

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

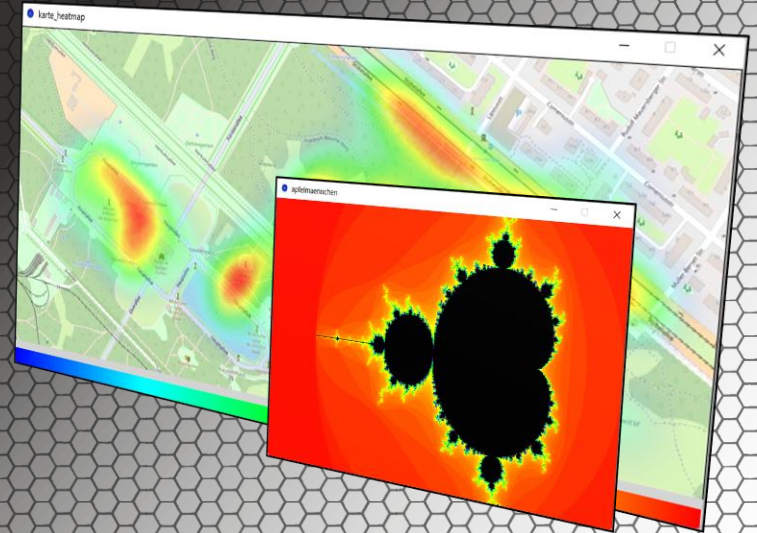
Sommersemester 2026



Punkte und Rastergrafiken

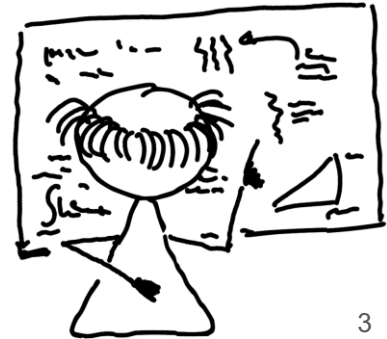
„Was wir sehen, ist niemals die Wirklichkeit selbst, sondern eine Interpretation.“ (Rudolf Arnheim)

PD Prof. Dr. Marco Block-Berlitz



Ziele und Inhalt

- Motivation zu Vektoren
- Vektoren als Richtungen
- Vektoren als Positionen
- Wechsel zwischen Richtungs- und Positionsdarstellung
- Vektorgrafiken, 3D-Modelle und Rastergrafiken
- Rastergrafiken und Pixel
- Positionen, Punkte oder Pixel
- Koordinatensystem Bildschirm
- Punkte ganzzahlig
- Punkte mit Gleitkommazahlen
- Kleines 2D-Malprogramm
- 2D-Rastergrafiken einlesen und anzeigen
- 2D-Rastergrafiken skalieren
- Aliasing-Effekte
- Konvolution und Blur-Effekt
- Heatmap erzeugen – Alpha Blending
- Mandelbrot- und Juliamenge
- Burning-Ship, Multibrot- und Newtonmengen
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben





„Äpfel lassen sich nicht mit Birnen vergleichen!“

Motivation zu Vektoren

Das ist einer der Gründe dafür, dass es Vektoren gibt [1]. Wollen wir die Anzahl der Äpfel von der Anzahl der Birnen getrennt betrachten, so erhalten wir ein **Paar von Zahlen**:



v_1 entspricht der **Anzahl der Äpfel**



v_2 entspricht der **Anzahl der Birnen**

Anstatt zwei Variablen zu schreiben, brauchen wir nur eine einzige zu notieren. Dieses Paar ist dann ein 2-dimensionaler **Vektor**:

$$\vec{v} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

Für einen Richtungsvektor $\vec{v}$ im 3D-Raum mit $\vec{v} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ und entsprechend im 2D-Raum mit $\vec{v} = \begin{bmatrix} x \\ y \end{bmatrix}$ werden die Achsenanteile immer an den gleichen Stellen des Vektors eingetragen.

[1] Strang G.: „Introduction to Linear Algebra“, 4. Auflage, Wellesley-Cambridge Press, 2009

[2] Videovorlesungen: https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/video_galleries/video-lectures/

Vektoren als Spaltenvektoren

Verständigen wir uns kurz auf die in dieser Veranstaltung verwendete Notation. Vektoren sind **immer als Spaltenvektoren** und in eckigen Klammern angegeben:

$$\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Sollten wir einen Spaltenvektor in einen **Zeilenvektor** umwandeln (transponieren), dann wird das mit einem T wie folgt notiert:

$$\vec{v}^T = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}^T = [v_1 \quad v_2 \quad \dots \quad v_n]$$

Sollten wir aus Platzgründen Vektoren im Text beschreiben, verwenden wir die **Zeilenschreibweise von Spaltenvektoren** $\vec{v} = (v_1, v_2, \dots, v_n)$ in runden Klammern mit $v_i \in \mathbb{R}$.

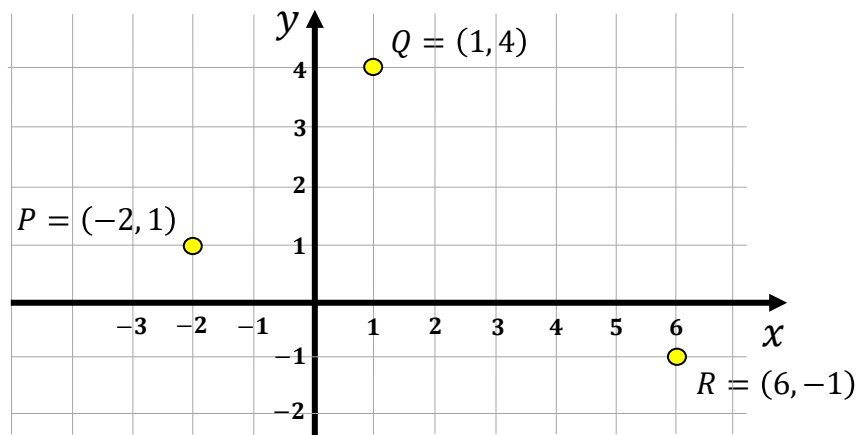
Aber Achtung: Der Vektor $\vec{v} = (v_1, v_2, \dots, v_n)$ ist trotz der verwendeten Zeilenschreibweise immer noch ein **Spaltenvektor** und muss entsprechend behandelt werden.

Vektoren als Positionen

Die Notation von Punkten $L, P, Q, \dots$ wird für Vektoren verwendet, die **Positionen** repräsentieren. Einen Punkt geben wir mit einem Großbuchstaben in Tupelschreibweise an:

$$P = (p_1, p_2, \dots, p_n)$$

Schauen wir uns verschiedene Punkte mit ihren Achsenanteilen im **kartesischen Koordinatensystem** an:



Die Positionen der Einträge im Tupel **sind immer fest**:

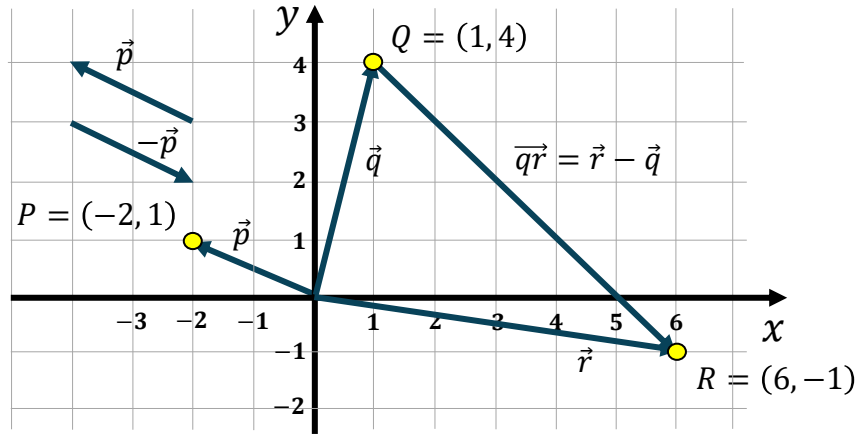
Für eine Position P im 3D-Raum mit $P = (x, y, z)$ werden die Achsenanteile analog zu den Vektoren immer an den gleichen Stellen eingetragen.



Wechsel zwischen Richtungs- und Positionsdarstellung

Wir können jederzeit zwischen einer **Richtungs- und Positionsdarstellung wechseln**. So können wir bspw. den Vektor zwischen zwei gegebenen Punkten P und Q über $\vec{p} = P - Q$ oder den Punkt V mit $V = \vec{v}$ angeben, der sich am Ende des Vektors $\vec{v}$ befindet, wenn der Vektor $\vec{v}$ im Ursprung startet.

Kleiner Tipp: Die Eselsbrücke **Köpfchen minus Schwänzchen** verrät leicht, in welche Richtung der resultierende Vektor $\vec{p} = P - Q$ zeigt.

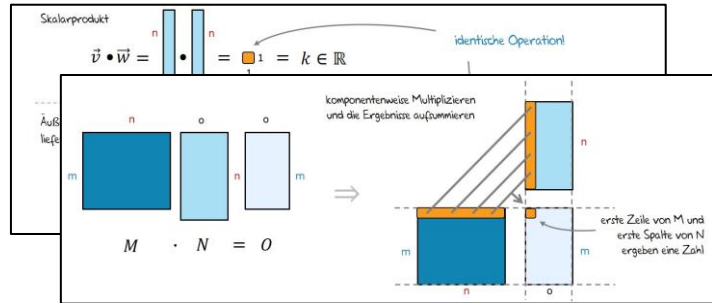
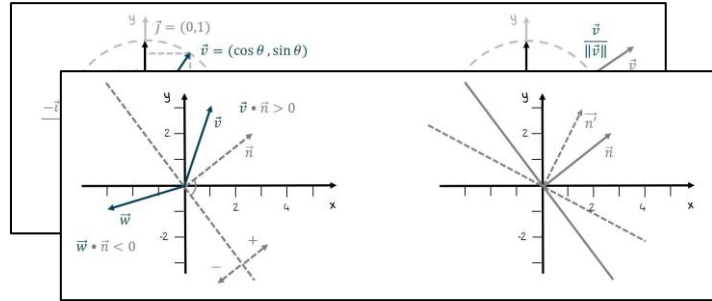


Hier sehen wir auch, dass die Zeilenschreibweise eines Spaltenvektors $\vec{v} = (v_1, v_2, \dots, v_n)$ in die natürliche Darstellung einer Position mit $V = (v_1, v_2, \dots, v_n)$ wechselt.

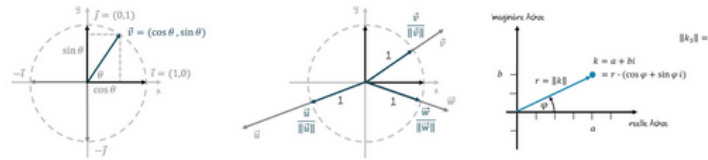


Learning "on demand"

Auf der Webseite www.vividus-verlag.de zum Buch [1] stehen zwei Kapitel zu den Themen Vektoren und Matrizen zum Download zur Verfügung.



Probekapitel

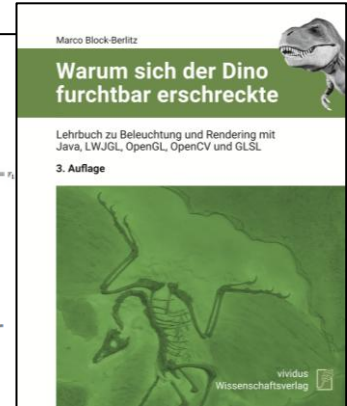


Um Beispiele für den Buchinhalt zu geben, haben wir **Probekapitel** zur Verfügung gestellt:

KAPITEL 4: Vektoren als Positionen, Richtungen und Farben [PDF 2.37 MB]

KAPITEL 9: Grundlegender Umgang mit Matrizen [PDF 2.12 MB]

Beide Kapitel stellen mit Vektoren und Matrizen wichtige Werkzeuge für die Computergrafik dar.



[1] Block-Berlitz, Marco: "Warum sich der Dino furchtbar erschreckte - Lehrbuch zu Beleuchtung und Rendering mit Java, LWJGL, OpenGL, OpenCV und GLSL", 3. Auflage, ISBN: 978-3-9819362-9-2, vividus Wissenschaftsverlag, 2021



Was haben die Vektoren mit
Computergrafik zu tun?

Vektorgrafiken, 3D-Modelle und Rastergrafiken

In der Computergrafik beschreiben wir **Vektorgrafiken** oder **3D-Modelle** und beschäftigen uns mit der Übertragung dieser Modelle auf verschiedene Ausgabegeräte. Das Ergebnis ist eine **Rastergrafik**.



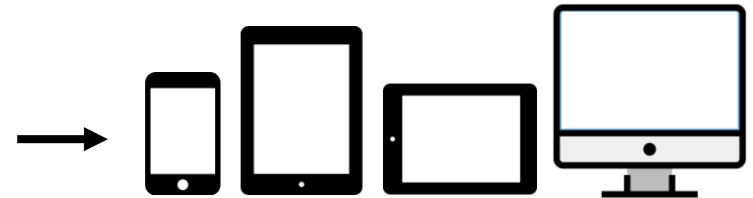
3D-Punkte eines
3D-Modells



Vermaschtes, texturiertes
3D-Modell

← 3D-Modellierung
3D-Rekonstruktionen [1]

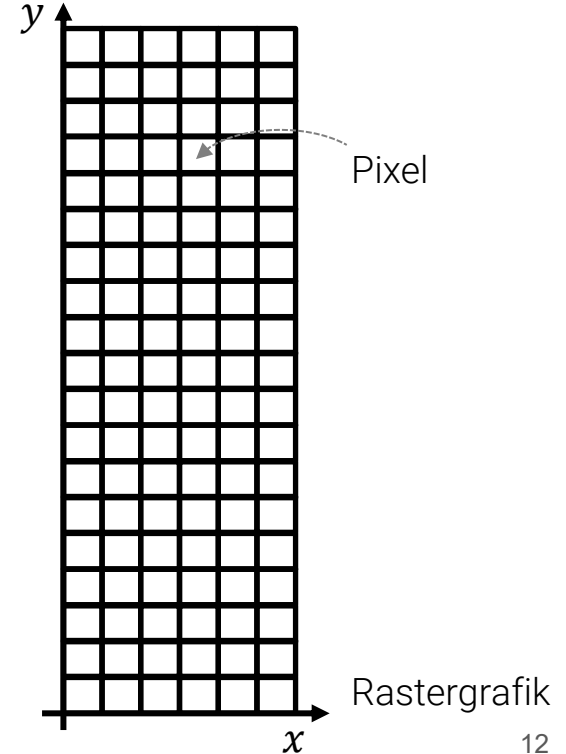
Ausgabegeräte mit verschiedenen
Bildaupflösungen: 1080p, 4k, ...



[1] <https://www.capturingreality.com>

Rastergrafiken und Pixel

Für die Erstellung einer **Rastergrafik** ist die Bildauflösung und damit die Anzahl der Zielpixel entscheidend (Pixel für *picture element*).

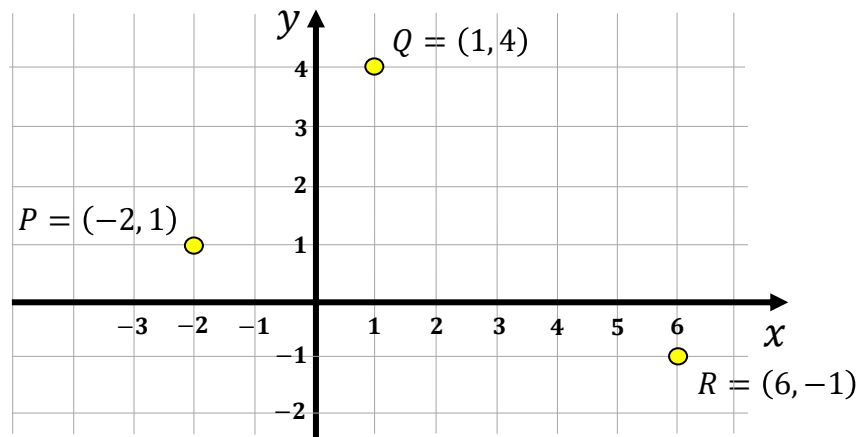


Positionen, Punkte oder Pixel

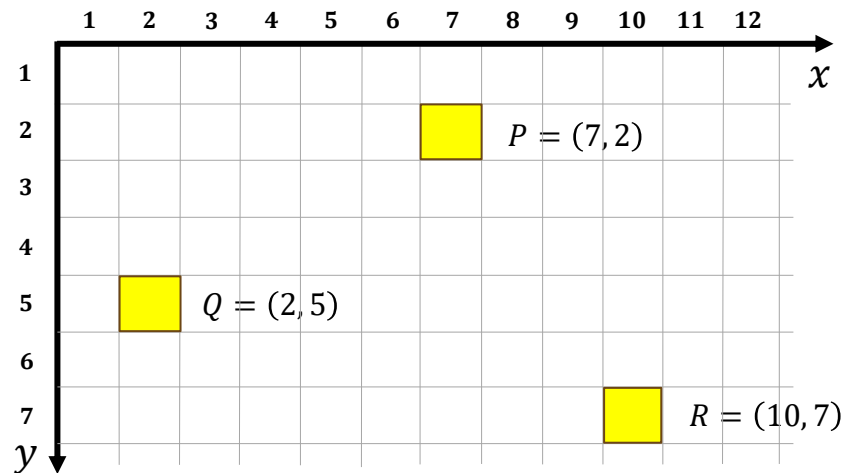
Ein **Pixel** (oder einen Punkt, eine Position) im Bild geben wir mit einem Großbuchstaben $L, P, Q, \dots$ in Tupelschreibweise an, wobei p_i die i -te Dimension darstellt:

$$P = (p_1, p_2, \dots, p_n)$$

Am häufigsten werden wir 2D-Punkte mit $P = (x, y)$ und 3D-Punkte mit $P = (x, y, z)$ verwenden. Schauen wir uns dazu kurz verschiedene 2D-Punkte mit ihren Achsenanteilen im **kartesischen Koordinatensystem** an:



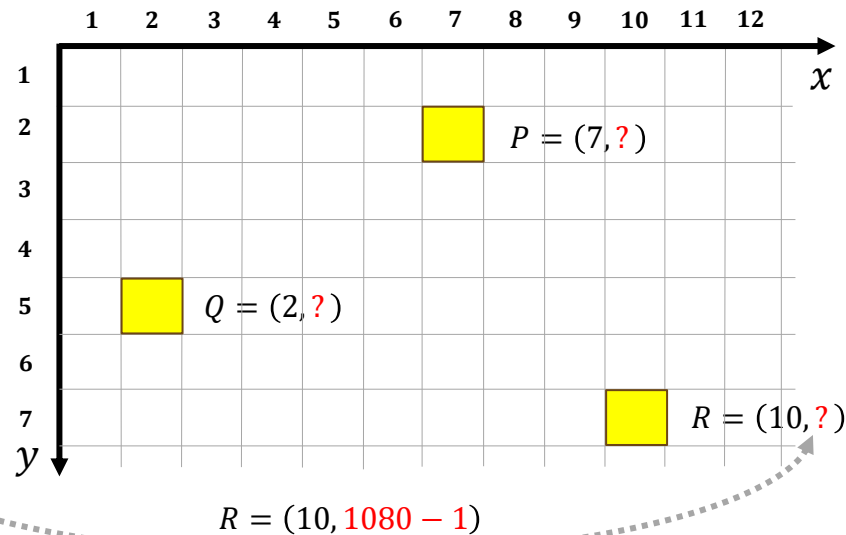
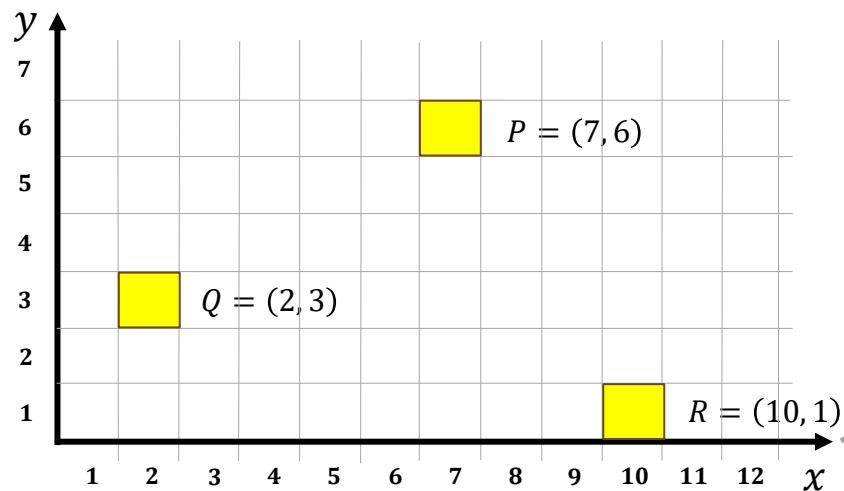
Mathematik



Informatik (Bildschirmkoordinaten)

Koordinatensystem Bildschirm

Das [Koordinatensystem auf einem Bildschirm](#) verwendet oft die y -Achse von oben nach unten. Was könnten Gründe dafür sein?



Die x -Koordinate bleibt erhalten, lediglich die y -Koordinate muss angepasst werden mit $p'_y = y_{max} - p_y$. Wir bestimmen also zunächst die Bildschirmauflösung (z. B. 1080p = 1920x1080).



Dann beginnen wir jetzt damit, einen Punkt zu zeichnen.

Punkte ganzzahlig

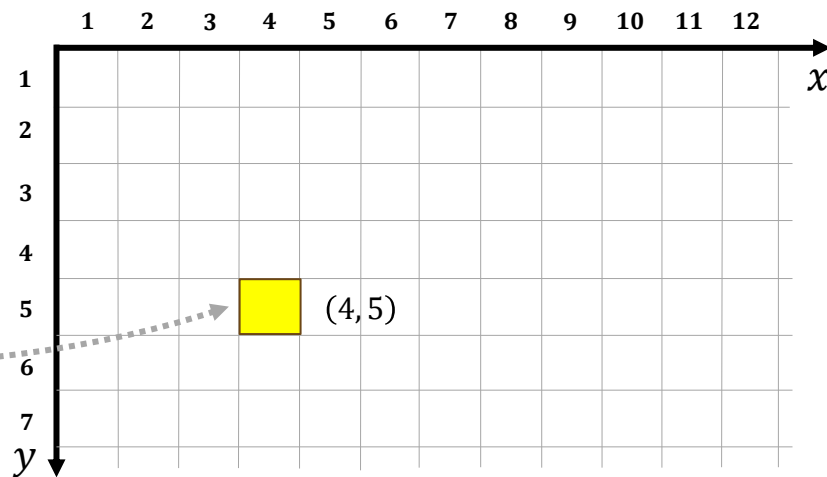
Wir können **Pixel** über unsere Funktion **point** in Processing aktivieren:

```
point(float x, float y)  
point(float x, float y, float z)
```

Processing

Jetzt können wir bspw. das Pixel $P = (4,5)$ anschalten:

```
point(4, 5);
```



Das klappt für ganzzahlige Pixel analog zum Raster.

Punkte mit Gleitkommazahlen

Wenn wir [Gleitkommazahlen](#) mit dieser Funktion darstellen wollen, werden diese nach einem bekannten Schema angepasst:

```
float x = 5.1;  
float y = 2.9;  
point(x, y);
```

Intern wird zu den Pixelwerten **0.5** addiert und der Nachkommateil abgeschnitten:

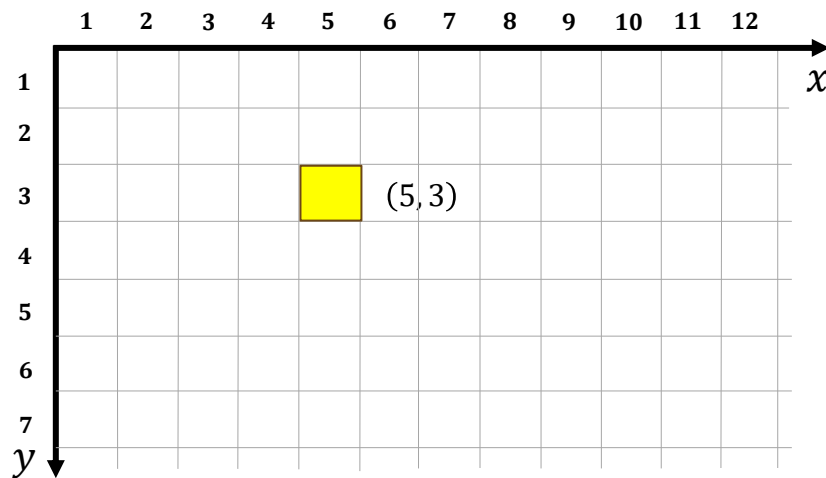
```
floor(x + 0.5)  
floor(y + 0.5)
```

Processing

Dadurch erhalten wir wieder einen regulären Rasterpunkt:

```
point(5, 3);
```

Wir haben oft Rundungs- und Entscheidungsaufgaben im Kontext der [Subpixel-Rendering-Problematik](#).





Mit Punkten lässt sich schon eine Menge anfangen ...

Kleines 2D-Malprogramm

Im unteren Bildbereich bereiten wir eine Farbpalette vor. Dort wählen wir eine **Farbe** aus und können direkt ein Pixel in den Bildbereich setzen.

```
color currentColor = color(0);
int pointSize      = 10;
int paletteHeight  = 40;
color[] palette = {
    color(0), /*Schwarz*/ color(255, 0, 0), color(0, 255, 0),
    color(0, 0, 255), color(255, 255, 0), color(255, 0, 255),
    color(0, 255, 255), color(255) /*Weiß*/
};

void setup() {
    size(800, 600);
    background(150);
}

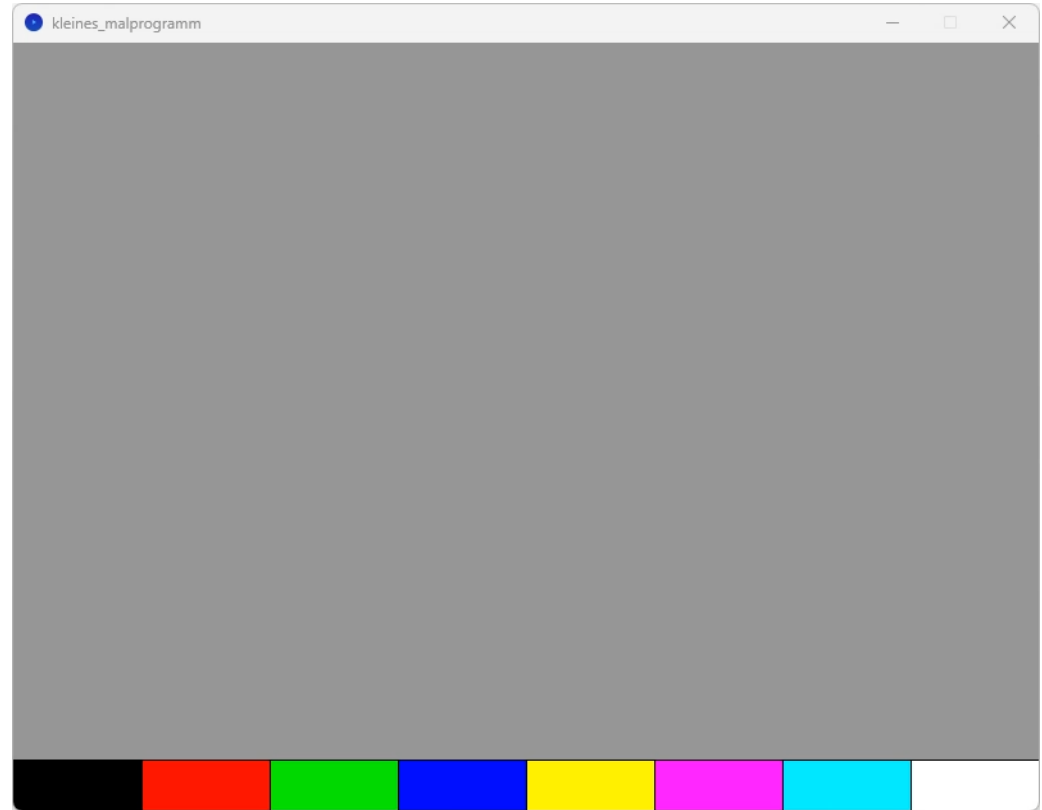
void draw() {
    drawPalette();
    // Nur zeichnen, wenn innerhalb der Zeichenfläche geklickt wird
    if (mousePressed && mouseY < height - paletteHeight) {
        noStroke();
        fill(currentColor);
        rect(mouseX - pointSize/2, mouseY - pointSize/2, pointSize, pointSize);
    }
}
```

```
void mousePressed() {
    // Klick auf Farbpalette?
    if (mouseY >= height - paletteHeight) {
        int w = width / palette.length; // Breite Farbfeld
        int index = mouseX / w; // Welches Feld?
        if (index >= 0 && index < palette.length) {
            currentColor = palette[index];
        }
    }
}

void drawPalette() {
    int w = width / palette.length;
    for (int i = 0; i < palette.length; i++) {
        fill(palette[i]);
        stroke(0);
        rect(i * w, height - paletteHeight, w, paletteHeight);
    }
}
```

Kleines 2D-Malprogramm

Wir können über Mausereignisse bereits einfache interaktive Visualisierungen, wie das vorgestellte Malprogramm, realisieren.



2D-Rastergrafiken (Bilder, Fotos) einlesen und anzeigen

Um ein Bild in Processing zu verarbeiten, müssen wir es über **Sketch->Datei hinzufügen** den Daten hinzufügen. Es wird ein Ordner **data** angelegt und die Datei entsprechend hineinkopiert.

Wir können es auslesen und den Farbwert direkt ohne Glättung oder andere Effekte anzeigen:

```
PImage img;  
  
void setup() {  
  size(960, 540);  
  img = loadImage("paphos.jpg");  
  img.loadPixels();  
  
  for (int i = 0; i < img.pixels.length; i++) {  
    int x = i % img.width;  
    int y = i / img.width;  
    set(x, y, img.pixels[i]);  
  }  
}
```

Bilder werden im Speicher oft **eindimensional** gehalten.

kapitel01/bild_direkt_anzeigen



paphos.jpg mit 960x540 Pixel



2D-Rastergrafiken (Bilder, Fotos) einlesen und anzeigen

Der **Java-Renderer** glättet **Punkte** durch kreisförmige Darstellungen (Funktion **point**), dadurch gibt es eine Überlagerung mit dem Hintergrund.

```
PImage img;  
  
void setup() {  
  size(960, 540);  
  img = loadImage("paphos.jpg");  
  img.loadPixels();  
  background(255);  
  
  for (int i = 0; i < img.pixels.length; i++) {  
    int x = i % img.width;  
    int y = i / img.width;  
    stroke(img.pixels[i]);  
    point(x, y);  
  }  
}
```

Das Ergebnis wirkt heller und unscharf.

kapitel01/bild_ueber_point_anzeigen



paphos.jpg mit 960x540 Pixel



2D-Rastergrafiken skalieren – Originalgröße anzeigen

In der Praxis müssen wir 2D-Rastergrafiken oft [skalieren](#). Dazu wollen wir zunächst das Bild im Original anzeigen:

```
PImage img;  
  
void setup() {  
  size(1015, 480);  
  img = loadImage("orchontal.jpg");  
  image(img, 0, 0);  
  println("Originalgröße: " + img.width + "x" + img.height);  
}
```



orchontal.jpg mit 1015x480 Pixel



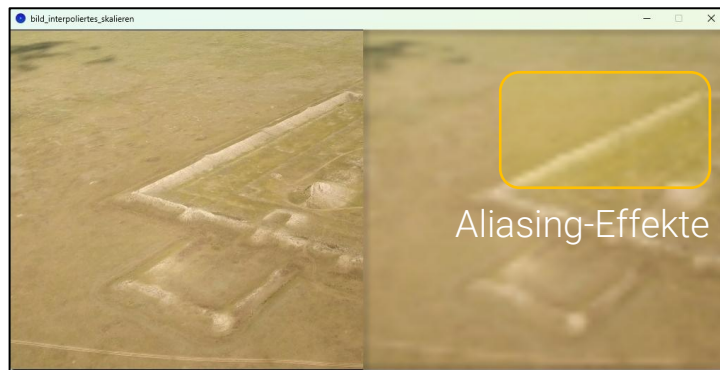
2D-Rastergrafiken skalieren – Naives Sampling

Jetzt wollen wir eine **verkleinerte Version des Bildes** erzeugen und diese dann wieder auf die Originalgröße bringen:

```
PImage img;  
  
void setup() {  
  size(1015, 480);  
  img = loadImage("orchontal.jpg");  
  
  // links Original  
  image(img, 0, 0);  
  
  // rechts skaliert, ohne Interpolation  
  PImage small = img.copy();  
  small.resize(img.width/8, img.height/8); // stark verkleinert  
  small.resize(img.width, img.height);      // wieder vergrößert  
  image(small, 500, 0);  
}
```



orchontal.jpg mit 1015x480 Pixel



Aliasing-Effekte

Aliasing entsteht, wenn ein hochaufgelöstes Bild oder Signal mit zu geringer Abtastrate erfasst oder dargestellt wird. Dabei gehen Informationen verloren, und es entstehen sichtbare Störungen oder falsche Muster:

- Treppeneffekte (Zacken) bei diagonalen Linien
- Moiré-Muster bei feinen Strukturen
- Flimmern bei bewegten oder dünnen Mustern
- Blockige oder harte Übergänge bei Nearest-Neighbor-Downsampling

Zu den klassischen Strategien (**Anti-Aliasing**) zur Verringerung von Aliasing gehören:

- Pre-Filtering/Blur-Effekt vor dem Downsampling (glättet hohe Frequenzen vorher)
- Bilineare/Bikubische Interpolation statt NN-Sampling (vermischt Pixelinformationen währenddessen)
- Supersampling/Multisampling für Glättung beim Rendern (in höherer Auflösung rendern, dann verkleinern)
- Verwendung von MIP-Mapping bei Texturen in 3D-Grafik (nimm Textur, die passt)

2D-Rastergrafiken skalieren – Naives Sampling Custom

Wir können die Verkleinerung des Bildes auch schnell selbst implementieren. Hier nutzen wir das [NN-Downsampling](#):

```
PImage img, scaled;

void setup() {
  size(2030, 480);
  img = loadImage("orchontal.jpg");
  int sampling = 2;
  scaled = createImage(img.width/sampling, img.height/sampling, RGB);
  img.loadPixels();
  scaled.loadPixels();

  // Einfaches Nearest-Neighbor Downsampling
  for (int y = 0; y < scaled.height; y++) {
    for (int x = 0; x < scaled.width; x++) {
      int srcX = x * sampling;
      int srcY = y * sampling;
      scaled.pixels[y*scaled.width + x] = img.pixels[srcY*img.width + srcX];
    }
  }

  scaled.updatePixels();
  image(img, 0, 0);           // Original
  image(scaled, img.width, 0); // um Faktor sampling verkleinertes Bild
}
```



orchontal.jpg mit 1015x480 Pixel

Grundidee:

$$I_{small}(x, y) = I(\text{round}(x \cdot s_x), \text{round}(y \cdot s_y))$$

$$s_x = W / W_{small} \quad s_y = H / H_{small}$$



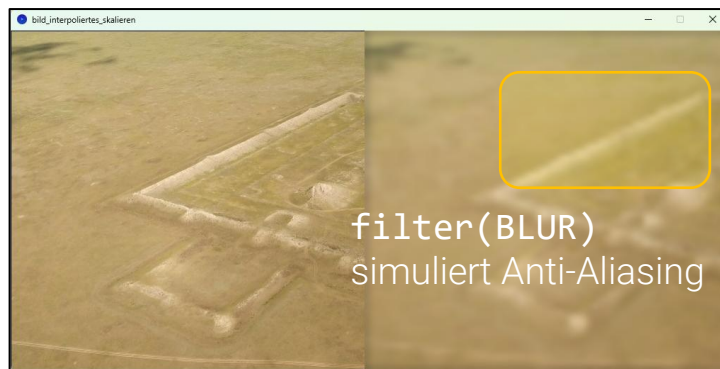
2D-Rastergrafiken skalieren – Mit Interpolation

Mit einem **Blur-Filter** können wir das Bild glätten und damit den Aliasing-Effekt reduzieren:

```
PImage img;  
  
void setup() {  
  size(1015, 480);  
  img = loadImage("orchontal.jpg");  
  
  // links Original  
  image(img, 0, 0);  
  
  // rechts skaliert, mit Interpolation  
  PImage small = img.copy();  
  small.resize(img.width/8, img.height/8); // stark verkleinert  
  small.resize(img.width, img.height);     // wieder vergrößert  
  small.filter(BLUR, 3);                   // Glättung  
  image(small, 500, 0);  
}
```

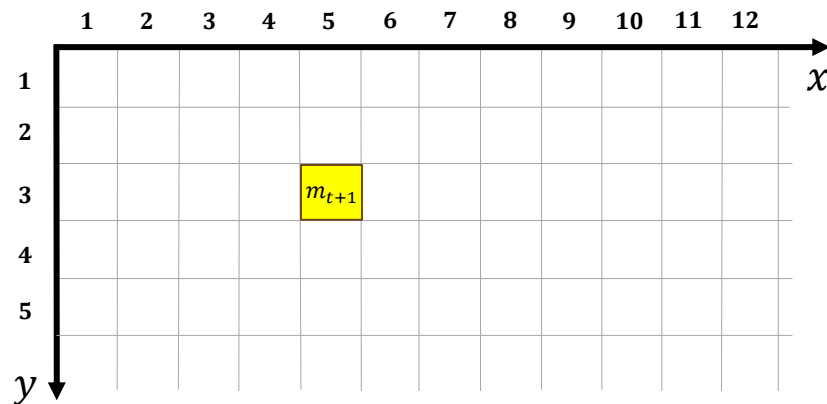
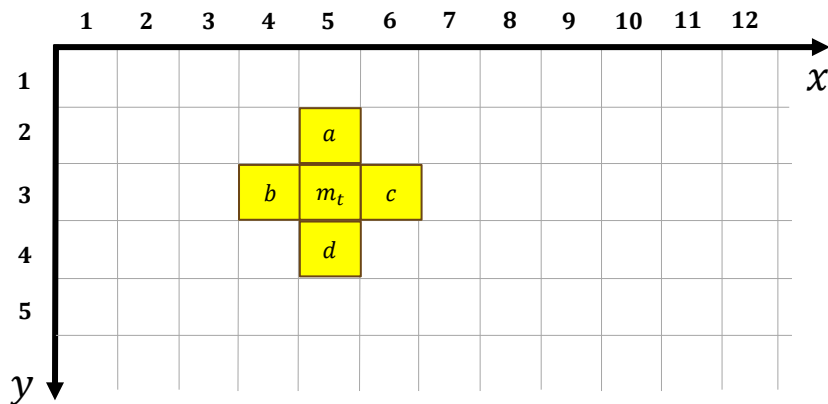


orchontal.jpg mit 1015x480 Pixel



Konvolution und Blur-Effekt

Die **Konvolution** ist ein sehr häufig eingesetztes Verfahren in der Bildverarbeitung mit sehr unterschiedlichen Anwendungsgebieten. Dabei wird aus einem gegebenen Bild ein neues erstellt und es werden für jedes Pixel Informationen über seine Nachbarn herangezogen, um die neue Pixelfarbe zu bestimmen.



Beim **Box-Blur** berechnet sich der neue Wert m_{t+1} aus der Summe der Nachbarn und des Ursprungspixels, geteilt durch die Anzahl der berücksichtigten Pixel: $m_{t+1} = (a + b + c + d + m_t) \cdot 0.2$

Bei **filter(BLUR, 3)** wird ein Kernel mit Radius 3 (7x7) erstellt und verwendet. Er reduziert hohe Frequenzen und wird daher als **Tiefpassfilter** bezeichnet. Alternative: Gaußfilter verwenden mit **filter(GAUSSIAN, 6)**.

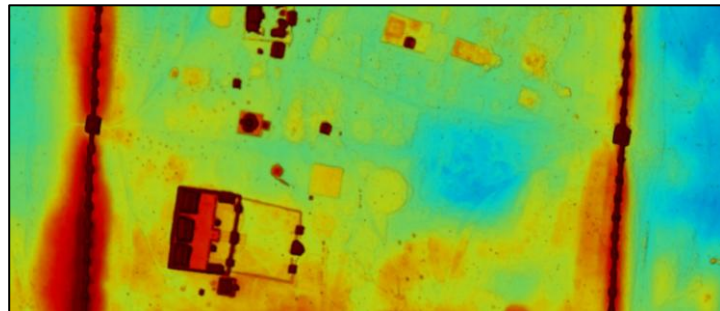


Kann ich die Fenstergröße auch nachträglich
an die Bildgröße anpassen?

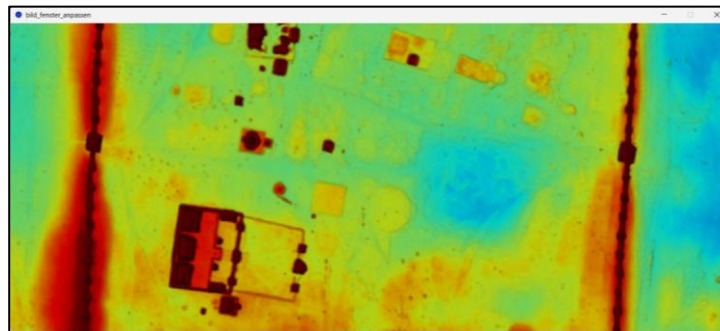
2D-Rastergrafiken skalieren – Originalgröße anzeigen

Um die Fenstergröße an die Bildgröße anzupassen, müssen wir zunächst das Bild vollständig laden. Am besten realisieren wir das in der Methode **settings**.

```
PImage img;  
  
void settings() {  
  img = loadImage("erdeene_zuu.jpg");  
  size(img.width, img.height);  
}  
  
void setup() {  
  image(img, 0, 0);  
}
```



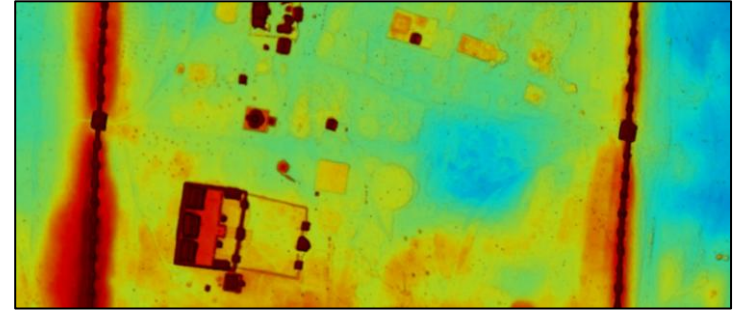
erdeene_zuu.jpg mit 1280x547 Pixel



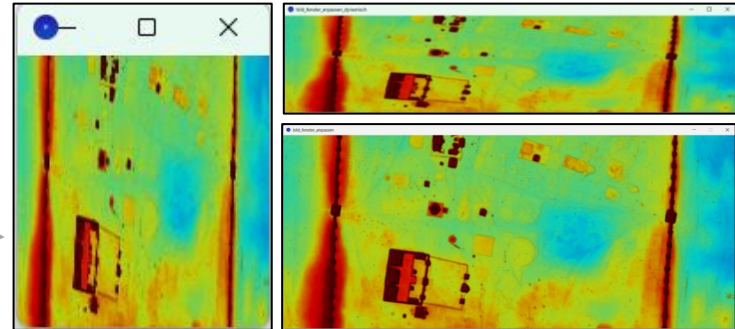
2D-Rastergrafiken skalieren – Originalgröße anzeigen

Um die Fenstergröße an die Bildgröße anzupassen, müssen wir zunächst das Bild vollständig laden. Am besten realisieren wir das in der Methode **settings**.

```
PImage img;  
  
void settings() {  
  img = loadImage("erdeene_zuu.jpg");  
  size(img.width, img.height);  
}  
  
void setup() {  
  surface.setResizable(true);  // Fenster darf geändert werden  
}  
  
void draw() {  
  background(0);  
  
  // Bild immer auf aktuelle Fenstergröße skalieren  
  image(img, 0, 0, width, height);  
}
```



erdeene_zuu.jpg mit 1280x547 Pixel



Heatmap erzeugen – Alpha-Blending

Zur Visualisierung von Intensitätsfeldern verwenden wir **Alpha-Blending**:

```
void renderHeatToImage() {  
    // ... finde maxVal in intensity[][]  
    // ... bei zu kleinen Werten soll alpha=0 gesetzt werden  
  
    heatImg.loadPixels();  
    for (int y=0; y<h; y++) {  
        for (int x=0; x<w; x++) {  
            float v = constrain(intensity[y][x]/maxVal, 0, 1); // normalisiert  
            int col = heatmapColor(v);  
            int alpha = (int)(v*230); // Transparenz proportional  
            int idx = y*w + x;  
            heatImg.pixels[idx] = (alpha << 24) | (col & 0x00FFFFFF);  
        }  
    }  
    heatImg.updatePixels();  
}
```

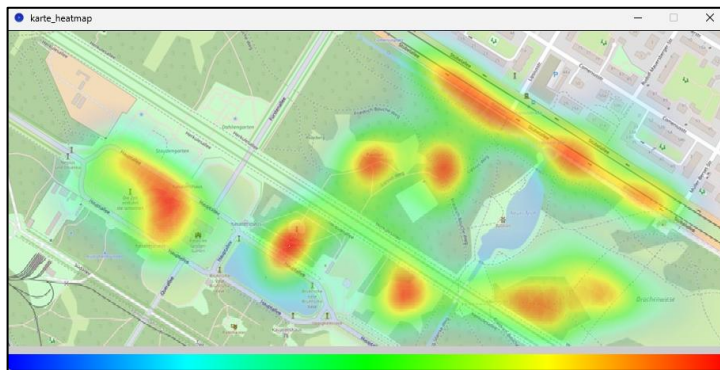
v=0 führt zu **alpha=0** (unsichtbar)

v=1 führt zu **alpha=230** (fast vollständig sichtbar)

kapitel01/karte_heatmap



karte.jpg mit 912x399 Pixel



Heatmap erzeugen – Alpha-Blending

Schauen wir uns dazu noch einmal kurz die einzelnen Schritte an:



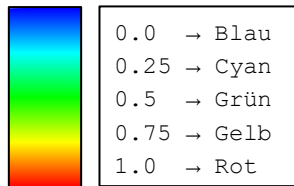
Ein Kartenbild liegt vor
912x399 Pixel

```
intensity[y][x] =  
0 1 2 1 0  
1 2 3 2 1  
2 3 5 3 2  
1 2 3 2 1  
0 1 2 1 0
```

Eine Wertematrix **intensity** liegt vor
maxVal = 5
Werte reichen von **0** bis **5**

```
intensity[y][x] =  
0.0 0.2 0.4 0.2 0.0  
0.2 0.4 0.6 0.4 0.2  
0.4 0.6 1.0 0.6 0.4  
0.2 0.4 0.6 0.4 0.2  
0.0 0.2 0.4 0.2 0.0
```

v = intensity[y][x]/maxVal
Jeder Wert wird auf **[0,1]**
normalisiert



v wird an **heatmapColor(v)**
übergeben. Jedes Pixel erhält
eine RGB-Farbe entsprechend
seiner Intensität.

```
alpha = v * 230
```

Kleiner Wert (transparent)
Hoher Wert (fast deckend)
Karte soll durchscheinen

```
idx = y * width + x
```

Processing speichert Bilder in
einem 1D-Array **pixels[]**
Zeilenweise von oben nach
unten, links nach rechts

```
pixels[idx] = (alpha << 24) | (col & 0x00FFFFFF)
```

Pixelwert **col** übernehmen und Alphawert
für **ARGB** setzen



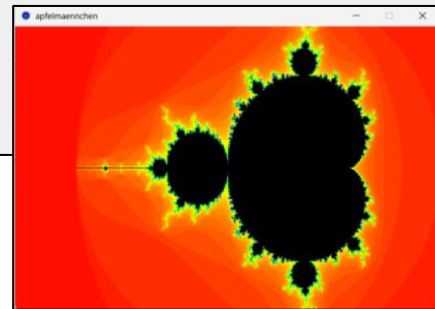
Fraktale in der Computergrafik

Ausschnitt der Mandelbrotmenge anzeigen

Die Mandelbrotmenge ist eine Menge komplexer Zahlen. Sie ist ein bekanntes Beispiel für ein Fraktal, das eine **Selbstähnlichkeit** aufweist und sich unendlich oft wiederholt, wenn man in die Details hineinzoomt.

```
void draw() {  
    loadPixels();  
    for (int px = 0; px < w; px++) {  
        for (int py = 0; py < h; py++) {  
            float x0 = map(px, 0, w, -2.5, 1);  
            float y0 = map(py, 0, h, -1, 1);  
            float x = 0;  
            float y = 0;  
            int iter = 0;  
  
            while (x*x + y*y <= 4 && iter < maxIter) {  
                float xtemp = x*x - y*y + x0;  
                y = 2*x*y + y0;  
                x = xtemp;  
                iter++;  
            }  
        }  
    }  
}
```

```
// Farbgebung: HSB, je nach Iteration  
float hue = iter / float(maxIter);  
float bright = iter < maxIter ? 1.0 : 0; // Punkte im  
                                         // Set schwarz  
  
int col = color(hue, 1, bright);  
int idx = py*w + px;  
pixels[idx] = col;  
}  
}  
updatePixels();  
}
```

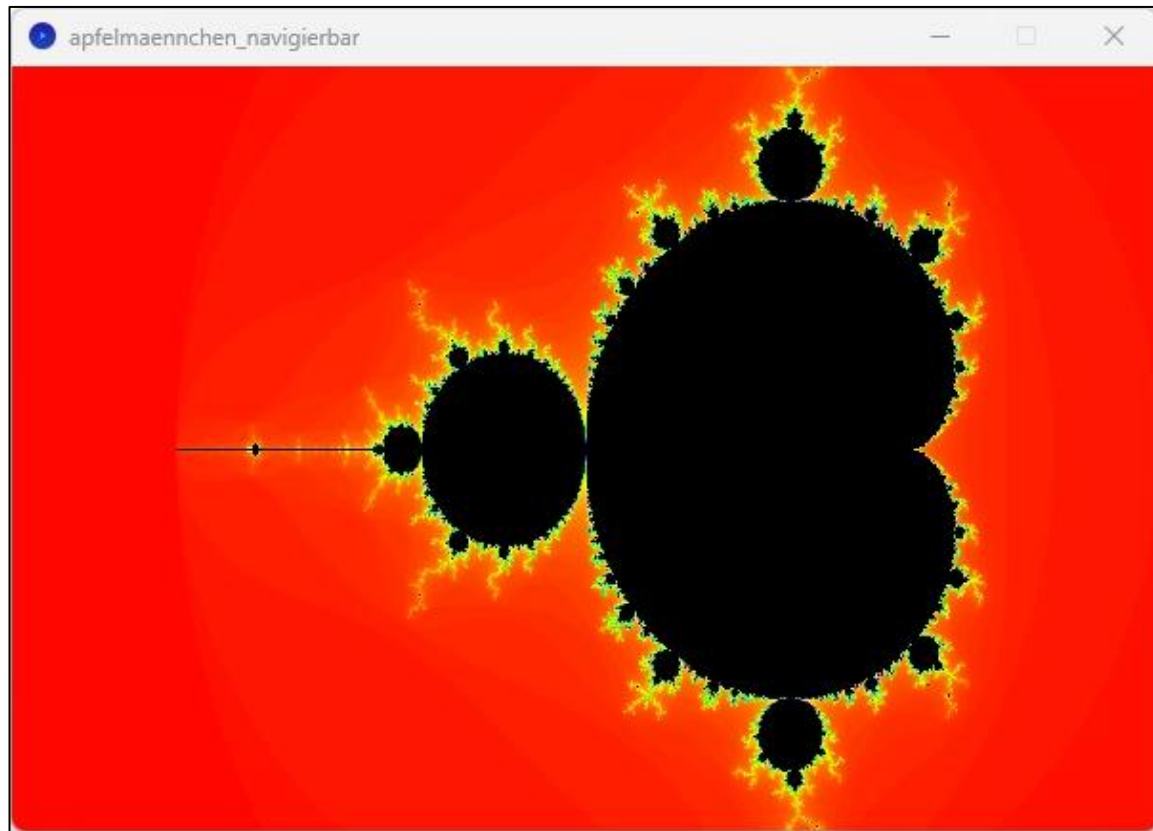


Jedes Pixel in eine komplexe Zahl c überführt. Für jeden Punkt iterieren wir $z_{n+1} = z_n^2 + c$, bis die Folge divergiert oder die maximale Iterationszahl erreicht ist. Der Wert **iter/maxIter** bestimmt die Farbgebung. Punkte, für die die Folge nicht divergiert, werden schwarz gezeichnet. Am Ende wird die Pixelfarbe gesetzt.

Mandelbrotmenge interaktiv

Die Mandelbrotmenge bietet sich für eine [interaktive Reise](#) an. Wir wollen dazu eine Lösung realisieren, die folgende Eigenschaften hat:

- **Zoom mit Mausexplorer:** Scroll nach oben = rein, nach unten = raus
- **Zoomzentrum = Mausposition:** der Punkt unter der Maus bleibt fix
- **Drag&Drop:** mit gedrückter Maustaste verschieben
- **Zoomabhängige Verschiebung:** Geschwindigkeit des Verschiebens passt sich Zoom an



Ausschnitt der Juliamenge anzeigen

Die Mandelbrotmenge ist eine Menge komplexer Zahlen. Sie ist ein bekanntes Beispiel für ein Fraktal, das eine **Selbstähnlichkeit** aufweist und sich unendlich oft wiederholt, wenn man in die Details hineinzoomt.

Die **Mandelbrotmenge** ergibt sich aus:

$$z_{n+1} = z_n^2 + c, \text{ mit } c \text{ ist variabel und } z_0 = 0$$

Die **Juliamenge** ergibt sich aus:

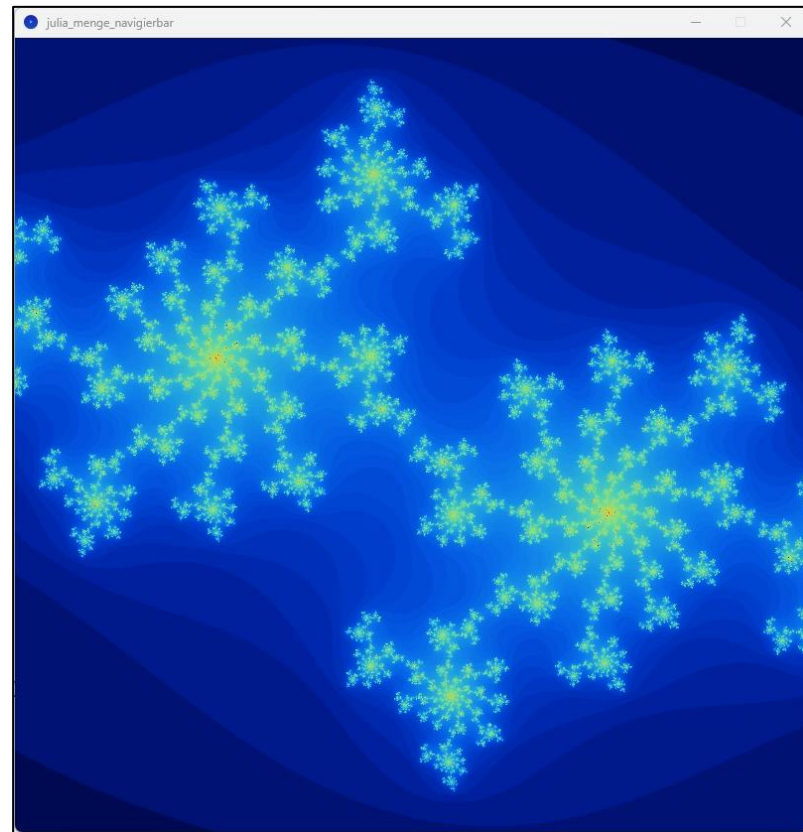
$$z_{n+1} = z_n^2 + c, \text{ mit } c \text{ ist fix und } z_0 \text{ ist der Pixelpunkt}$$



Juliamenge interaktiv

Die Juliamenge bietet sich ebenfalls für eine [interaktive Reise](#) an. Wir wollen dazu eine Lösung realisieren, die folgende Eigenschaften hat:

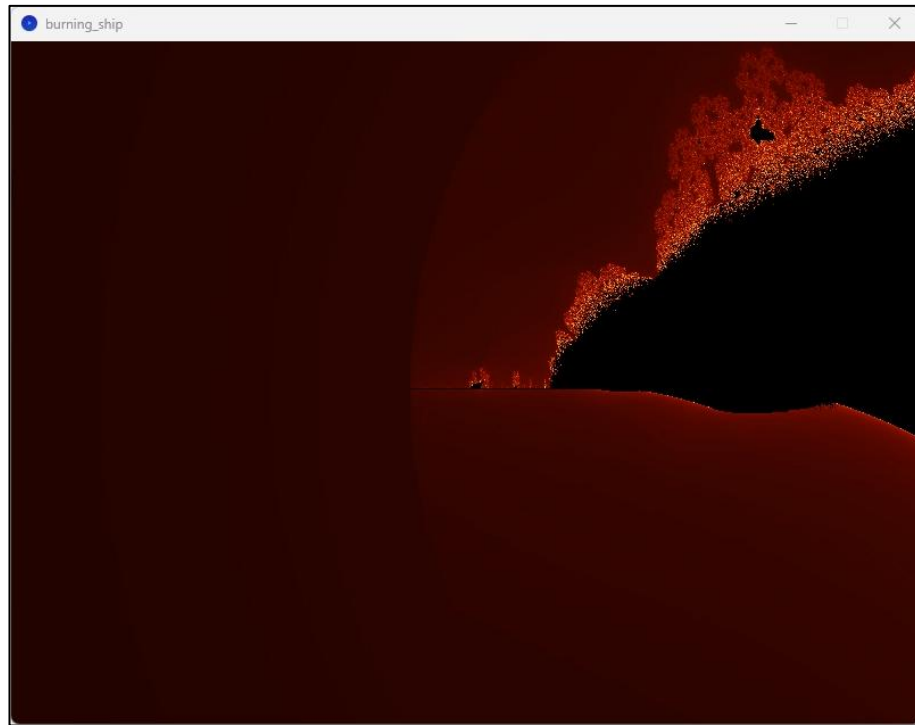
- **Zoom mit Mausrad:** Scroll nach oben = rein, nach unten = raus
- **Zoomzentrum = Mausposition:** der Punkt unter der Maus bleibt fix
- **Drag&Drop:** mit gedrückter Maustaste verschieben
- **Zoomabhängige Verschiebung:** Geschwindigkeit des Verschiebens passt sich Zoom an



Weitere interessante Fraktale: Burning-Ship

Burning-Ship [1]:

$$z_{n+1} = (|Re(z_n)| + i|Im(z_n)|)^2 + c$$



Der Unterschied zwischen dem Burning-Ship und der Mandelbrot-Menge besteht darin, dass die Real- und Imaginärteile vor der Quadrierung bei jeder Iteration auf ihre jeweiligen Absolutwerte gesetzt werden.

kapitel01/burning_ship

[1] Michelitsch M., Rössler O. E.: "The "Burning Ship" and Its Quasi-Julia Sets", In: Computers & Graphics Vol. 16, No. 4, pp. 435–438, 1992

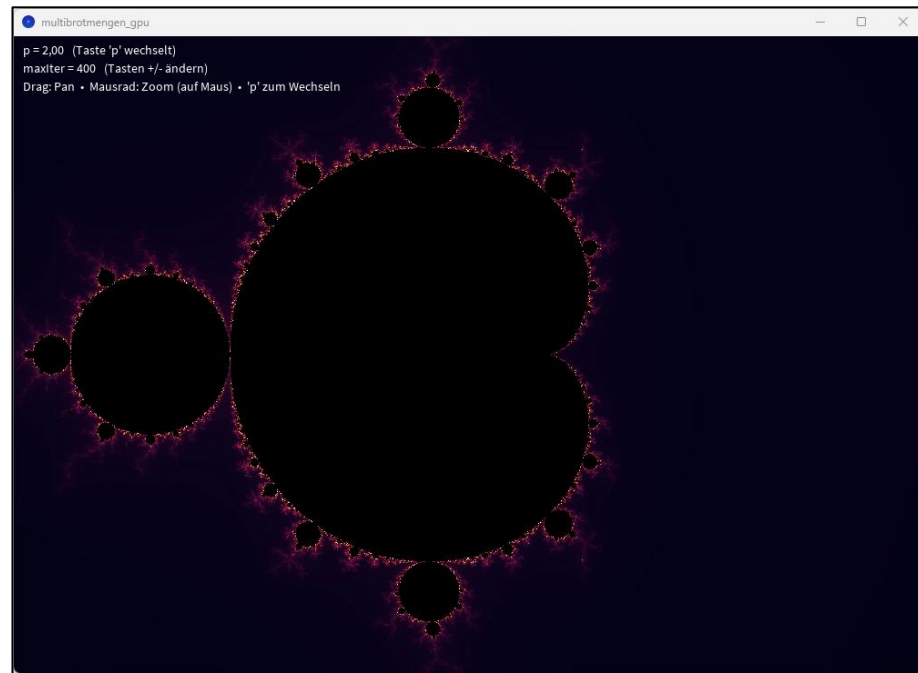
Weitere interessante Fraktale: Multibrotmengen

Burning-Ship:

$$z_{n+1} = (|Re(z_n)| + i|Im(z_n)|)^2 + c$$

Multibrotmengen ($p = 2, 3, 4, 5, 6 \dots$) [1]:

$$z_{n+1} = z_n^p + c$$



Der Unterschied zwischen dem Burning-Ship und der Mandelbrot-Menge besteht darin, dass die Real- und Imaginärteile vor der Quadrierung bei jeder Iteration auf ihre jeweiligen Absolutwerte gesetzt werden.

kapitel01/multibrotmengen_gpu

[1] https://en.wikipedia.org/wiki/Multibrot_set

Weitere interessante Fraktale: Newtonmengen

Burning-Ship:

$$z_{n+1} = (|Re(z_n)| + i|Im(z_n)|)^2 + c$$

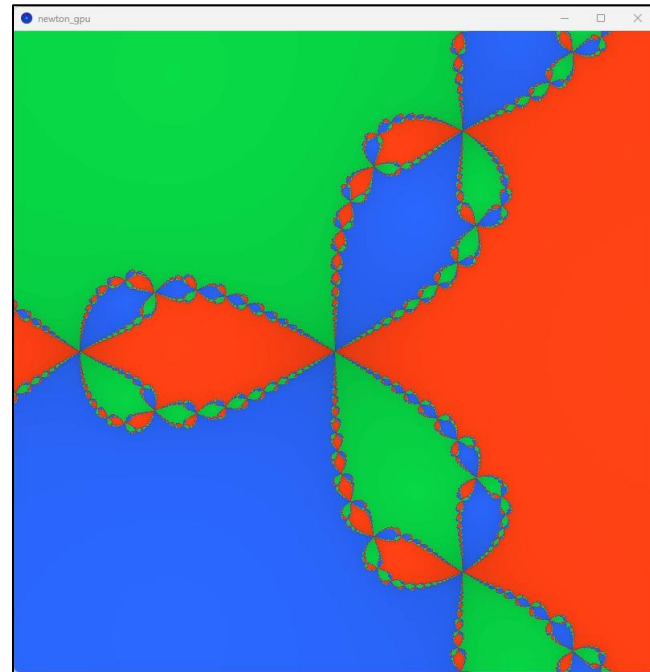
Multibrotmengen ($p = 2, 3, 4, 5, 6 \dots$):

$$z_{n+1} = z_n^p + c$$

Newtonmengen:

$$z_{n+1} = z_n - f(z_n)/f'(z_n)$$

z. B. für $f(z) = z^3 - 1$



Diese Funktion hat 3 Wurzeln (rot, grün, blau) die zu 0 führen. Wenn wir von einem beliebigen Punkt starten, zu welcher Wurzel führt das?

Punkte und Rastergrafiken

Key Takeaways

- Vektoren und Punkte (Gemeinsamkeiten/Unterschiede)
- Koordinatensysteme verstehen (Mathematik/Bildschirm)
- Rastergrafiken als Pixelraster begreifen (ARGB-Strukturen)
- Punkte zeichnen (Subpixel-Rendering-Problematik)
- Sampling und Aliasing (Blur, Tiefpassfilter)
- Praktische Arbeit mit Rastergrafiken (Heatmaps)
- Erste Schritte in die Welt der Fraktale



Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Was ist der Unterschied zwischen einem Punkt und einem Richtungsvektor?
- (2) Warum können wir zwischen Richtungs- und Positionsdarstellung wechseln?
- (3) Was ist bei der Umrechnung mathematischer Koordinaten in Bildschirmkoordinaten zu beachten?
- (4) Warum wird in Processing bei `point(x, y)` intern `floor(x + 0.5)` ausgeführt?
- (5) Was ist Aliasing und warum entsteht es beim Downsampling?
- (6) Ein Bild hat eine Auflösung von **1080p**. Wie viele Megabyte werden benötigt, wenn jeder Pixel ARGB-32-bit kodiert ist?
- (7) Erklären Sie den Unterschied zwischen Raster- und Vektorgrafiken anhand eines selbstgewählten Beispiels.
- (8) Beschreiben Sie Aliasing und nennen Sie zwei Maßnahmen, um es zu reduzieren.
- (9) Erläutern Sie das Prinzip des Alpha-Blending in eigenen Worten.

Praktische Aufgaben

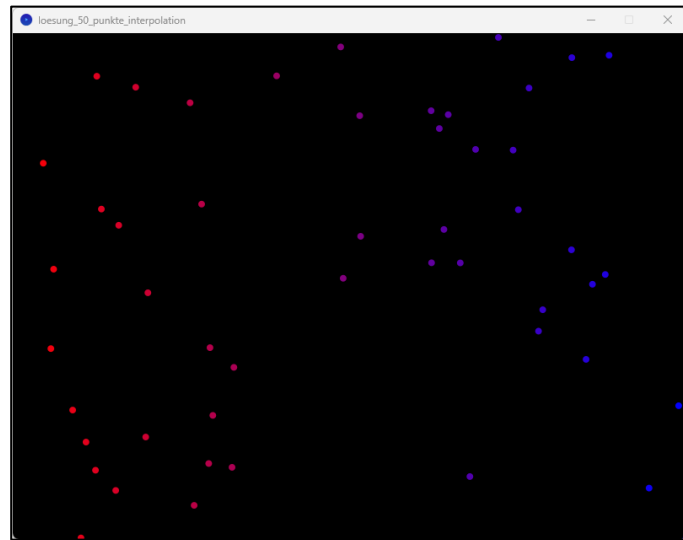
Die praktischen Aufgaben dienen zur Vertiefung des Stoffs und sollten selbstständig bearbeitet werden.

Aufgabe 1) [Processing] Zeichnen Sie **50** zufällige Punkte in ein Fenster der Größe **800x600** und färben Sie jeden Punkt anhand seiner x -Koordinate mit einem linearen RGB-Farbverlauf zwischen zwei vorgegebenen Farben ein.

Hinweis: Verwenden Sie die Methode `lerpColor`.

Beispiel:

```
color c1 = color(255, 0, 0);  
color c2 = color(0, 0, 255);
```



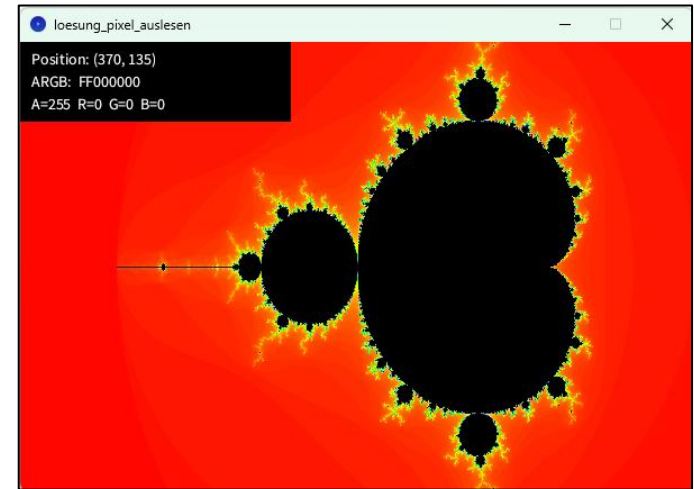
Praktische Aufgaben

Die praktischen Aufgaben dienen zur Vertiefung des Stoffs und sollten selbstständig bearbeitet werden.

Aufgabe 2) [Processing] Laden Sie ein Bild und visualisieren Sie es im Fenster. Implementieren Sie anschließend ein Pixel-Inspektionswerkzeug, das bei Bewegung der Maus über das Bild die folgenden Werte des jeweils darunterliegenden Pixels anzeigt:

- **Pixelposition** (x, y),
- den vollständigen **ARGB-Wert** (als Hexadezimalwert),
- sowie die einzelnen **Farbkanäle** A, R, G und B.

Die Werte sollen in einem kleinen Overlay-Feld im Fenster dargestellt oder alternativ als Zeichenkette in der Konsole ausgegeben werden.



Praktische Aufgaben

Die praktischen Aufgaben dienen zur Vertiefung des Stoffs und sollten selbstständig bearbeitet werden.

Aufgabe 3) [Processing] Mit der Phoenixmenge $z_{n+1} = z_n^2 + c + pz_{n-1}$ lassen sich ebenfalls sehr spektakuläre Fraktale erzeugen. Versuchen Sie sich doch einmal an einer Implementierung. Nutzen Sie dafür die vorhandenen Code-Beispiele aus.





Vielen Dank für die Aufmerksamkeit